Let's talk about Web Accessibility (A11y) & WCAG 2.1 For Angular

Sudeep Parchure | AI Assisted | May 2026 | Angular Solutions Architect

> Source code and working examples: github.com/sudeep31/angularTodo — documents folder

---

Table of Contents

---

1. The Problem in Brief

96.3% of the top one million websites have detectable WCAG failures (WebAIM 2024, avg. 56 errors/page). Over **1.3 billion people** globally have some form of disability. Accessibility failures are not edge cases — they exclude real users, carry legal liability (4,000+ US lawsuits/year), and degrade SEO.

Audience segment	Scale	What they need
Blind users	~36 million globally	Screen reader support (ARIA, semantics)
Low vision	~246 million globally	Contrast ≥4.5:1, resizable text, reflow
Motor impaired	~200 million globally	Keyboard-only navigation
Deaf / HoH	~430 million globally	Captions, transcripts, visual alerts
Cognitive differences	~1 billion globally	Predictable UI, clear errors, readable text

> **Legal mandates:** ADA + Section 508 (US), EN 301 549 / EAA effective 2025 (EU), Equality Act (UK), AODA (Canada), DDA (Australia) — all reference WCAG 2.1 Level AA.

---

2. WCAG 2.1 — The POUR Standard

WCAG 2.1 (W3C, June 2018) defines **78 success criteria** under four foundational principles — **POUR**.

Principle	Core Obligation	Key Sub-areas
P — Perceivable	All information and UI components must be presentable in ways users can perceive	Text alternatives for non-text content; captions and audio descriptions; adaptable presentation; distinguishable (contrast, colour, audio)
O — Operable	UI components and navigation must be operable by every user	Keyboard accessible, no traps; sufficient time; no seizure-inducing content; navigable with headings and focus order; pointer input alternatives
U — Understandable	Information and UI operation must be understandable	Readable (language identified); predictable behaviour; input assistance (error identification, suggestions)
R — Robust	Content must be robust enough for current and future assistive technologies	Valid, well-formed HTML; name, role, value for all UI components; status messages programmatically determined

Perceivable — key requirements

- **1.1.1 Non-text Content (A):** All images, icons, and non-text elements have a text alternative.
- **1.2.2 Captions (A):** Pre-recorded video includes captions.
- **1.3.1 Info and Relationships (A):** Structural relationships conveyed through semantic HTML or ARIA.
- **1.4.3 Contrast (AA):** Text has ≥4.5:1 contrast against its background (3:1 for large text).
- **1.4.4 Resize Text (AA):** Text resizable to 200% without loss of content or functionality.
- **1.4.10 Reflow (AA):** Content readable at 320 CSS px width without horizontal scroll.
- **1.4.11 Non-text Contrast (AA):** UI components and graphics have ≥3:1 contrast.

Operable — key requirements

- **2.1.1 Keyboard (A):** All functionality operable via keyboard alone.
- **2.1.2 No Keyboard Trap (A):** Focus can always be moved away using keyboard.
- **2.4.1 Bypass Blocks (A):** Skip links or landmark navigation to bypass repeated content.
- **2.4.3 Focus Order (A):** Navigation follows a logical, meaningful sequence.
- **2.4.7 Focus Visible (AA):** Keyboard focus is always visually indicated.
- **2.5.3 Label in Name (A):** Interactive elements' accessible name contains their visible label text.

Understandable — key requirements

- **3.1.1 Language of Page (A):** Page language declared with `lang` attribute.
- **3.2.1 On Focus (A):** Focus alone does not trigger unexpected context changes.
- **3.3.1 Error Identification (A):** Errors described to users in text, not just colour.
- **3.3.2 Labels or Instructions (A):** Input fields have visible labels and instructions.

Robust — key requirements

- **4.1.1 Parsing (A):** HTML is valid (no duplicate IDs, no unclosed tags).
- **4.1.2 Name, Role, Value (A):** Every UI component has an accessible name, role, and value.
- **4.1.3 Status Messages (AA):** Status messages programmatically determined without receiving focus.

---

3. Conformance Levels & Key Success Criteria

Level	Description	Legal / Industry Status
A	Minimum baseline — failing makes content inaccessible for some users	Required by most laws
AA	Standard target — addresses major barriers for most disability types	**Industry standard; legally mandated in EU, UK, US, Canada, Australia**
AAA	Enhanced support — not achievable for all content	Recommended for government, health, specialist services

Target: AA for every public-facing Angular application.

---

4. Semantic HTML — The Foundation

Native HTML elements carry built-in semantics — role, keyboard behaviour, and state management — that ARIA cannot fully replicate. Always prefer semantic HTML; use ARIA only for gaps it cannot fill.

Landmark elements

HTML Element	Implicit Role	One Per Page?*	Notes
`<header>` (body child)	`banner`	Yes	Site-level header
`<nav>`	`navigation`	No	Add `aria-label` when multiple navs exist
`<main>`	`main`	Yes	Target of skip link; use `tabindex="-1"`
`<section aria-labelledby>`	`region`	No	Needs accessible name to appear in landmark list
`<aside>`	`complementary`	No	Sidebars, related content
`<footer>` (body child)	`contentinfo`	Yes	Site-level footer
`<form aria-labelledby>`	`form`	No	Named forms appear as form landmarks
`<search>`	`search`	No	Search widgets

> * **One Per Page?** — `Yes` means only one instance of this element should exist per page. Duplicate `Yes` landmarks (e.g., two `<main>` or two `<header>`) break screen reader landmark navigation. `No` means multiple instances are valid and expected — use `aria-label` to distinguish them.

> **Component-level rule:** Angular components MUST use `role="region"` + `aria-label` in their `host` binding — never `role="main"`. Duplicate main landmarks break screen reader navigation.

Semantic element benefits

Use this	Not this	Why
`<button>`	`<div role="button">`	Keyboard support (Enter/Space), focus, disabled state — built in
`<a href>`	`<span role="link">`	Tab-focusable, correct announcement, right-click menu
`<input type="checkbox">`	`<div role="checkbox">`	Checked state, keyboard toggle, label association
`<h1>` – `<h6>`	`<div class="heading">`	Document outline; screen reader `H` key navigation
`<ul>` / `<ol>`	`<div>` lists	"List, N items" announcement; item count context
`<label for>`	`aria-label` on input	Click-to-focus; works without JavaScript

Skip link — mandatory (WCAG 2.4.1, Level A)

```
<!-- First focusable child of <body> -->
<a
  href="#main-content"
  class="sr-only focus:not-sr-only focus:fixed focus:top-4 focus:left-4 focus:z-50"
>
  Skip to main content
</a>

<main id="main-content" tabindex="-1">
  <!-- tabindex="-1" required: Safari does not focus <main> on skip link click without it -->
</main>
```

Semantic HTML skeleton (Angular app shell)

```
<!-- app.html -->
<a href="#main-content" class="skip-link">Skip to main content</a>

<header role="banner">
  <nav aria-label="Primary">…</nav>
</header>

<main id="main-content" tabindex="-1">
  <router-outlet />
</main>

<aside aria-label="Todo statistics">…</aside>

<footer role="contentinfo">
  <nav aria-label="Footer">…</nav>
</footer>

<!-- Global live regions — always in DOM, initially empty -->
<div role="status" aria-live="polite" aria-atomic="true" class="sr-only">{{ statusMsg() }}</div>
<div role="alert" aria-live="assertive" aria-atomic="true" class="sr-only">{{ errorMsg() }}</div>
```

---

5. ARIA — What It Is and When to Use It

WAI-ARIA (Accessible Rich Internet Applications) is a W3C specification that adds semantic metadata to dynamic, JavaScript-driven UI elements that have no native HTML equivalent.

ARIA communicates three things to assistive technologies:

ARIA Layer	Answers the question	Example attributes
Roles	What kind of element is this?	`role="dialog"`, `role="tablist"`, `role="combobox"`
Properties	What is its metadata / relationship?	`aria-label`, `aria-describedby`, `aria-required`
States	What is its current condition?	`aria-expanded`, `aria-checked`, `aria-invalid`

The Accessibility Tree

```
HTML DOM + ARIA
      ↓
Browser builds Accessibility Tree
(semantic model, no visual CSS)
      ↓
Platform Accessibility API
(Windows: UI Automation · macOS: NSAccessibility)
      ↓
Assistive Technology (NVDA, JAWS, VoiceOver, Braille display)
```

The accessibility tree is what screen readers read. Incorrect or missing ARIA corrupts this model — the screen reader announces incorrect or empty information.

The Five Rules of ARIA (W3C)

Rule	Requirement
1	Use native HTML if it does the job — do not add ARIA to what HTML already conveys
2	Do not change the native semantics of HTML elements (e.g., no `role` override on `<button>`)
3	If you add a role, implement its complete keyboard interaction contract
4	Never apply `aria-hidden="true"` to a focusable element
5	Every interactive element must have an accessible name

Hiding content: three techniques

Technique	Sighted users see it?	Screen reader reads it?	When to use
`aria-hidden="true"`	Yes	No	Decorative icons, duplicate text, visual-only elements
`display: none` / `[hidden]`	No	No	Hidden panels, closed modals, off-screen content
`.sr-only` CSS class	No	Yes	Context labels visible only to AT (e.g., "Delete: item name")

---

6. ARIA Roles Reference

Landmark roles

Screen reader users navigate by landmarks (NVDA: `D` key, JAWS: `R` key) — equivalent to a page-level table of contents.

Role	HTML Equivalent	One per page?*	`aria-label` required?	Announces as
`banner`	`<header>` (body child)	Yes	No	"Banner, landmark"
`navigation`	`<nav>`	No	When >1 nav	"Primary, navigation"
`main`	`<main>`	**Yes**	No	"Main, landmark"
`region`	`<section aria-labelledby>`	No	**Yes** (or invisible)	"Todo list, region"
`complementary`	`<aside>`	No	Recommended	"Todo statistics, complementary"
`contentinfo`	`<footer>` (body child)	Yes	No	"Contentinfo, landmark"
`form`	`<form aria-labelledby>`	No	**Yes**	"Add task, form"
`search`	`<search>`	No	Recommended	"Search, landmark"

> * **One per page?** — `Yes` means only one instance of this role should exist per page. Having duplicates (e.g., two `main` or two `banner` landmarks) creates ambiguity in the screen reader landmark list. `No` means multiple instances are valid — always add `aria-label` to distinguish them when more than one appears.

Widget roles

Widget roles define interactive controls. Each role carries a **keyboard contract** — failing to implement it means WCAG 2.1.1 violation.

Role	What it represents	Keyboard contract	Key required ARIA
`button`	Actionable control	Enter / Space to activate	Name; optionally `aria-pressed`, `aria-expanded`
`checkbox`	Binary or tri-state toggle	Space to toggle	`aria-checked`: `'true'` / `'false'` / `'mixed'`
`radio`	Exclusive selection in group	Arrow keys within `radiogroup`	`aria-checked`, inside `role="radiogroup"`
`switch`	On/off toggle	Space to toggle	`aria-checked`: `'true'` / `'false'` (not `'mixed'`)
`slider`	Range value	Arrow keys; Home/End	`aria-valuenow`, `aria-valuemin`, `aria-valuemax`
`textbox`	Single-line text input	Standard text editing	`aria-multiline` if multi-line; name required
`combobox`	Text input + listbox	Arrow Down opens; Enter selects; Escape closes	`aria-expanded`, `aria-controls` → listbox
`listbox`	Selection list	Arrow keys; Home/End; Space/Enter select	`aria-multiselectable` if multiple
`option`	Item inside listbox/combobox	Managed by parent listbox	`aria-selected`
`tab`	Tab selector	Arrow Left/Right within tablist	`aria-selected`, `aria-controls` → tabpanel
`tablist`	Container for tabs	—	`aria-label` or `aria-labelledby`
`tabpanel`	Content pane	Tab to enter from active tab	`aria-labelledby` → active tab
`dialog`	Modal overlay	Escape to close; Tab trapped inside	`aria-labelledby`, `aria-modal="true"`
`alertdialog`	Modal requiring response	Escape, focus auto-moves inside	`aria-labelledby`, `aria-describedby`
`menu`	Floating menu	Arrow Down/Up; Enter selects; Escape closes	`aria-labelledby`
`menuitem`	Item in menu	Managed by parent menu	Name
`tooltip`	Supplemental hint	Shown on hover and focus	`role="tooltip"`, linked via `aria-describedby`
`tree`	Hierarchical list	Arrow keys; Space/Enter; Home/End	`aria-expanded`, `aria-selected`
`grid`	Interactive table	Arrow keys for cell navigation	`aria-rowcount`, `aria-colcount`

Document structure roles

Used to add semantics to presentational markup without implying interactivity.

Role	Purpose	Example
`heading`	Heading — use `aria-level`	Custom heading component
`img`	Image region with description	Composite SVG figures
`list` / `listitem`	Ordered/unordered list	CSS-styled custom lists
`group`	Logical grouping	Radio group, set of related controls
`note`	Supplemental information	Side note, caveat
`definition`	Term definition	Glossary entries
`none` / `presentation`	Strips element semantics	Layout tables, decorative wrappers

> `role="none"` **usage:** Apply to layout tables and nav `<ul>` elements styled as flex bars to remove spurious "list" announcements. Never apply to focusable elements.

Live region roles

Role	Equivalent ARIA	Politeness	Use for
`alert`	`aria-live="assertive"` + `aria-atomic="true"`	Assertive (interrupts)	Errors, session timeouts, destructive confirmations
`status`	`aria-live="polite"` + `aria-atomic="true"`	Polite (waits)	Save confirmations, filter counts, info messages
`log`	`aria-live="polite"`	Polite (sequential)	Chat feeds, activity logs, append-only content
`marquee`	`aria-live="off"`	Off	Tickers — use sparingly
`timer`	`aria-live="off"`	Off	Clocks, countdowns — expose only on demand

---

7. ARIA Properties Reference

Properties describe metadata that rarely changes during a session — name, description, and relationships between elements.

Naming properties (highest-to-lowest priority)

Property	Priority	Use for	AT announcement
`aria-labelledby="id1 id2"`	1 (highest)	Sections, dialogs — link to visible heading text	Concatenates referenced element text
`aria-label="string"`	2	Icon buttons, controls with no visible text label	Inline string read as name
`<label for="id">`	3	All form inputs — the preferred native approach	Label text read before input role
Element text content	4	Buttons, links with text children	Text content is the name
`title="string"`	5 (lowest)	Last resort; unreliable across screen readers	Read only if no other name source

```
<!-- Naming priority examples -->
<label for="task-title">Task title</label>
<input id="task-title" type="text" aria-required="true" />
<!-- AT: "Task title, text field, required" -->

<button [attr.aria-label]="'Delete: ' + todo().title">
  <svg aria-hidden="true">…</svg>
</button>
<!-- AT: "Delete: Buy groceries, button" -->

<h2 id="filter-heading">Filter Tasks</h2>
<section aria-labelledby="filter-heading">…</section>
<!-- AT: "Filter Tasks, region" -->
```

> **Icon button with no name (WCAG 4.1.2 failure):**
> `<button><svg aria-hidden="true"/></button>` → AT announces "button" for every item.
> Fix: `aria-label="Delete: [item title]"` + `aria-hidden="true"` on the SVG.

Description properties

Property	Purpose	AT announcement timing
`aria-describedby="id"`	Points to longer contextual description	After name + role
`aria-details="id"`	Points to complex description (figure, code block)	Navigable separately

```
<input id="task-title" aria-required="true" aria-invalid="true" aria-describedby="title-error" />
<p id="title-error" role="alert">Title cannot be empty</p>
<!-- AT: "Task title, text field, required, invalid — Title cannot be empty" -->
```

Relationship properties

Property	What it declares	Typical use
`aria-controls="id"`	"I control the content in that element"	Button → collapsible panel
`aria-owns="id"`	"That element is logically my child (DOM aside)"	Virtual tree, custom dropdown
`aria-haspopup="menu|listbox|dialog|grid|tree"`	"Activating me opens a popup of this type"	Menu buttons, comboboxes
`aria-flowto="id"`	"Reading order continues there instead of DOM order"	Complex layouts

Widget properties

Property	Values	AT effect
`aria-required`	`'true'`	Announces "required" when field is focused
`aria-readonly`	`'true'`	Announces "read only" — field still focusable
`aria-multiselectable`	`'true'`	Announces "multiple selection possible"
`aria-multiline`	`'true'`	Announces "multi-line" for textbox
`aria-placeholder`	String	Read when field is empty (not a label substitute)
`aria-valuemin/max/now`	Number	Announces slider/spinbutton current and range values
`aria-valuetext`	String	Human-readable override for `aria-valuenow`
`aria-orientation`	`'horizontal'|'vertical'`	Arrow key direction for sliders, toolbars, menus
`aria-setsize`	Number	"Item X of N" in virtualised lists
`aria-posinset`	Number	Item position within a set

Live region properties

Property	Values	Effect
`aria-live`	`'off'|'polite'|'assertive'`	Sets announcement timing for region updates
`aria-atomic`	`'true'|'false'`	`true`: announce entire region content; `false`: only changed nodes
`aria-relevant`	`'additions'|'removals'|'text'|'all'`	What types of DOM change trigger announcement
`aria-busy`	`'true'|'false'`	Suppresses announcements while `true` — release after data loads

> **`aria-atomic="true"` matters:** Without it, a region like `<span class="count">5</span> items remaining` may announce only `"4"` when count changes. With `aria-atomic="true"` it announces `"4 items remaining"` — full context preserved.

---

8. ARIA States Reference

States describe runtime conditions that change as users interact. They update the accessibility tree immediately, and screen readers pick up the change in real time.

Widget states

State	Values	AT announcement	Usage notes
`aria-expanded`	`'true'|'false'`	"expanded" / "collapsed"	Accordions, menus, dropdowns, comboboxes
`aria-checked`	`'true'|'false'|'mixed'`	"checked" / "unchecked" / "partially checked"	Checkboxes, `role="switch"`, "select all"
`aria-pressed`	`'true'|'false'|'mixed'`	"pressed" / "not pressed"	Toggle buttons (bold, mute, pin)
`aria-selected`	`'true'|'false'`	"selected"	`role="tab"`, `role="option"`, `role="treeitem"`
`aria-disabled`	`'true'|'false'`	"unavailable" / "dimmed"	Keeps element focusable (unlike native `disabled`)
`aria-hidden`	`'true'|null`	Element excluded from AT	Decorative, duplicate, or off-screen content
`aria-invalid`	`'true'|'false'|'grammar'|'spelling'`	"invalid entry"	Form fields after failed validation
`aria-busy`	`'true'|'false'`	"busy"	While async content loads
`aria-grabbed`	`'true'|'false'|null`	"grabbed"	Drag-and-drop (deprecated in ARIA 1.1 — use custom)

`aria-disabled` vs native `disabled`

	Native `disabled`	`aria-disabled="true"`
Keyboard reachable?	No (removed from tab order)	Yes (still focusable)
AT announces it?	No (element invisible to AT)	Yes ("unavailable")
Form submission includes value?	No	Yes (handle in JS)
Use when	Element is fully inactive	Element is visible but pending a prerequisite

Document structure states

State	Values	Purpose
`aria-current`	`'page'|'step'|'location'|'date'|'time'|'true'`	Mark the current item in a set (nav link, breadcrumb step)
`aria-errormessage`	`id`	Points to error text element (pair with `aria-invalid="true"`)

Form validation pattern

```
// Angular 21 — signal-based
protected readonly titleInvalid = computed(() =>
  this.errors().title ? 'true' : null   // null removes the attribute
);
protected readonly titleErrorId = 'task-title-error';
```

```
<input
  id="task-title"
  [attr.aria-invalid]="titleInvalid()"
  [attr.aria-describedby]="titleInvalid() ? titleErrorId : null"
  aria-required="true"
/>

<p [id]="titleErrorId" role="alert" [hidden]="!titleInvalid()">Title is required</p>
<!-- AT on focus when invalid: "Task title, text field, required, invalid — Title is required" -->
```

> **Always bind `null` to remove attributes — not `'false'`.**
> `[attr.aria-invalid]="false"` → adds attribute `aria-invalid="false"` → AT announces "invalid: false" as noise.
> `[attr.aria-invalid]="null"` → removes the attribute entirely → silent when valid.

Accordion / expandable panel pattern

```
@Component({
  selector: 'app-accordion-item',
  host: { '[attr.aria-expanded]': 'isOpen()' },
})
export class AccordionItemComponent {
  protected readonly isOpen = signal(false);
  toggle(): void {
    this.isOpen.update((v) => !v);
  }
}
```

```
<button [attr.aria-expanded]="isOpen()" [attr.aria-controls]="panelId">{{ title() }}</button>
<div [id]="panelId" [hidden]="!isOpen()" role="region" [attr.aria-labelledby]="buttonId">
  <ng-content />
</div>
<!-- AT on button: "Section title, button, collapsed" / "expanded" -->
```

Tab panel state pattern

```
<div role="tablist" aria-label="Todo filters">
  @for (tab of tabs(); track tab.id) {
  <button
    role="tab"
    [attr.aria-selected]="activeTab() === tab.id"
    [attr.aria-controls]="'panel-' + tab.id"
    [id]="'tab-' + tab.id"
    [tabindex]="activeTab() === tab.id ? 0 : -1"
    (click)="setTab(tab.id)"
    (keydown)="onTabKey($event)"
  >
    {{ tab.label }}
  </button>
  }
</div>
@for (tab of tabs(); track tab.id) {
<div
  role="tabpanel"
  [id]="'panel-' + tab.id"
  [attr.aria-labelledby]="'tab-' + tab.id"
  [hidden]="activeTab() !== tab.id"
>
  <ng-container *ngTemplateOutlet="tab.content" />
</div>
}
```

---

9. Live Regions & Dynamic Announcements

Without live regions, all dynamic UI changes — saves, adds, deletes, filter results — are completely silent to screen reader users. The user submits a form and hears nothing; the task is added and nothing is announced.

`aria-live` politeness levels

Value	When screen reader announces	Use for
`polite`	After the current utterance finishes	Saves, additions, filter changes, success messages
`assertive`	**Immediately — interrupts everything**	Deletions, critical errors, session expirations
`off`	Never (manual access only)	Timers, tickers — announce only on demand

> **Overusing `assertive` degrades UX.** Every small action interrupting the screen reader mid-sentence is like constant pop-up notifications. Reserve `assertive` for destructive or urgent events only.

The timing rule — most common mistake

The live region element must exist in the DOM before its content is injected.

```
<!-- WRONG: region created at same time as content — most screen readers ignore it -->
@if (message()) {
<div role="status">{{ message() }}</div>
}

<!-- CORRECT: region always present; content changes inside it -->
<div role="status" class="sr-only" aria-atomic="true">@if (message()) { {{ message() }} }</div>
```

`aria-atomic` — announce full context

```
<!-- aria-atomic="false" (default) — may announce only "4" when count changes -->
<div aria-live="polite"><span class="count">{{ count() }}</span> items remaining</div>

<!-- aria-atomic="true" — always announces full string: "4 items remaining" -->
<div aria-live="polite" aria-atomic="true">{{ count() }} items remaining</div>
```

Angular CDK LiveAnnouncer — recommended approach

```
import { LiveAnnouncer } from '@angular/cdk/a11y';
import { inject, Component } from '@angular/core';

@Component({
  /* standalone, OnPush */
})
export class TodoListComponent {
  private readonly liveAnnouncer = inject(LiveAnnouncer);

  onAdd(title: string): void {
    this.todoService.add(title);
    this.liveAnnouncer.announce(`"${title}" added`, 'polite');
  }

  onDelete(title: string, id: string): void {
    this.todoService.delete(id);
    // Assertive — destructive action, must confirm immediately
    this.liveAnnouncer.announce(`"${title}" deleted`, 'assertive');
  }

  onFilterChange(filter: string): void {
    const count = this.filteredTodos().length;
    this.liveAnnouncer.announce(
      `Showing ${count} ${filter} task${count !== 1 ? 's' : ''}`,
      'polite',
    );
  }
}
```

CDK manages the underlying live region element, handles browser-specific quirks, and cleans up on destroy.

SPA route change announcements

Angular Router does not notify screen readers when the URL changes. The page content changes silently.

```
// a11y-title.strategy.ts
import { Injectable } from '@angular/core';
import { TitleStrategy, RouterStateSnapshot } from '@angular/router';
import { Title } from '@angular/platform-browser';
import { LiveAnnouncer } from '@angular/cdk/a11y';

@Injectable({ providedIn: 'root' })
export class A11yTitleStrategy extends TitleStrategy {
  private readonly title = inject(Title);
  private readonly liveAnnouncer = inject(LiveAnnouncer);

  override updateTitle(snapshot: RouterStateSnapshot): void {
    const title = this.buildTitle(snapshot) ?? 'Angular App';
    this.title.setTitle(title);
    this.liveAnnouncer.announce(`Navigated to ${title}`, 'polite');
  }
}

// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes), { provide: TitleStrategy, useClass: A11yTitleStrategy }],
};
```

---

10. Complex Widget Patterns (APG)

Before building any custom interactive widget, consult the **ARIA Authoring Practices Guide (APG)**: `w3.org/WAI/ARIA/apg/patterns`

The APG defines the complete contract for each widget: required roles, expected keyboard interactions, and focus management behaviour.

Modal dialog

```
<div
  role="dialog"
  aria-modal="true"
  aria-labelledby="dialog-title"
  aria-describedby="dialog-desc"
  cdkTrapFocus
  cdkTrapFocusAutoCapture
>
  <h2 id="dialog-title">Confirm Delete</h2>
  <p id="dialog-desc">This action cannot be undone.</p>
  <button (click)="confirm()">Delete</button>
  <button (click)="close()">Cancel</button>
</div>

<!-- Make background inert while dialog is open -->
<main [attr.inert]="isModalOpen() ? '' : null">…</main>
```

Focus management rules:

- On open: focus moves to the first interactive element inside the dialog
- `Tab` / `Shift+Tab` cycles within the dialog only
- `Escape` closes the dialog
- On close: **return focus to the exact element that opened the dialog**

> **`aria-modal="true"` is not a substitute for `inert`.**
> `aria-modal` tells *screen readers* to ignore background content. The native `inert` attribute also prevents keyboard focus from reaching background elements — both are required for a complete implementation.

Tabs (APG roving tabindex pattern)

Key	Behaviour
`Tab`	Enter tablist — lands on active tab only
`Arrow Left / Right`	Switch between tabs (wraps at ends)
`Home`	First tab
`End`	Last tab
`Tab` from active tab	Move into the tab panel content

```
onTabKey(event: KeyboardEvent): void {
  const tabs = this.tabs();
  const current = tabs.findIndex(t => t.id === this.activeTab());
  if (event.key === 'ArrowRight') {
    event.preventDefault();
    this.setTab(tabs[(current + 1) % tabs.length].id);
  } else if (event.key === 'ArrowLeft') {
    event.preventDefault();
    this.setTab(tabs[(current - 1 + tabs.length) % tabs.length].id);
  } else if (event.key === 'Home') {
    event.preventDefault(); this.setTab(tabs[0].id);
  } else if (event.key === 'End') {
    event.preventDefault(); this.setTab(tabs[tabs.length - 1].id);
  }
}
```

> `event.preventDefault()` is mandatory on arrow keys — without it, the browser scrolls the page while the user navigates tabs.

Combobox

```
<div class="combobox-wrapper">
  <label for="task-search" id="combo-label">Search tasks</label>
  <input
    id="task-search"
    role="combobox"
    [attr.aria-expanded]="isOpen()"
    aria-haspopup="listbox"
    aria-controls="combo-listbox"
    aria-autocomplete="list"
    [attr.aria-activedescendant]="activeOption()"
  />
  <ul id="combo-listbox" role="listbox" aria-labelledby="combo-label">
    @for (opt of filtered(); track opt.id) {
    <li role="option" [id]="'opt-' + opt.id" [attr.aria-selected]="opt.id === selected()">
      {{ opt.label }}
    </li>
    }
  </ul>
</div>
```

Key interactions: `Arrow Down` opens listbox; `Arrow Down/Up` moves `aria-activedescendant`; `Enter` selects; `Escape` closes without selecting; `Tab` accepts current value.

Tooltip pattern

```
<!-- Trigger: always a focusable element -->
<button
  type="button"
  [attr.aria-describedby]="isTooltipVisible() ? 'tip-save' : null"
  (mouseenter)="showTooltip()"
  (mouseleave)="hideTooltip()"
  (focus)="showTooltip()"
  (blur)="hideTooltip()"
>
  Save
</button>

<!-- Always in DOM — shown/hidden via CSS, not @if -->
<div role="tooltip" id="tip-save" [hidden]="!isTooltipVisible()">
  Mark this todo as complete to remove it from the active list.
</div>
```

> Tooltips must appear on **both focus and hover** (WCAG 1.4.13). The trigger must be a focusable element. Never put interactive content inside a tooltip.

---

11. Screen Reader Reference

Market share (WebAIM 2024)

Screen reader	Share	Platform	Cost
JAWS	~46%	Windows	Commercial
NVDA	~30%	Windows	Free
VoiceOver iOS	~11%	iPhone / iPad	Built-in
VoiceOver macOS	~9%	macOS	Built-in
TalkBack	~7%	Android	Built-in
Narrator	~2%	Windows 11	Built-in

Minimum test matrix (covers ~85% of real users)

Screen reader	Browser	Why this pairing
NVDA	Firefox	Most common among AT users; free; strong ARIA support
JAWS	Chrome / Edge	Enterprise standard; different virtual buffer behaviour
VoiceOver	Safari	Apple's only supported pairing — essential for iOS parity

NVDA keyboard reference

Key	Action
`Ctrl+Alt+N`	Launch NVDA
`NVDA+F7`	List all page landmarks
`NVDA+F6`	List all headings
`D`	Jump to next landmark
`H`	Jump to next heading
`B`	Jump to next button
`F`	Jump to next form field
`Tab`	Jump to next interactive element
`NVDA+Space`	Toggle browse / forms mode
`NVDA+Q`	Quit NVDA

ARIA behaviour across screen readers

ARIA	NVDA + Firefox	JAWS + Chrome	VoiceOver macOS
`aria-expanded="false"`	"collapsed"	"collapsed"	"collapsed"
`aria-expanded="true"`	"expanded"	"expanded"	"expanded"
`aria-invalid="true"`	"invalid entry"	"invalid entry"	"invalid"
`role="alert"` fires	Interrupts	Interrupts	Interrupts
`aria-live="polite"`	After speech	After speech	After speech
`aria-current="page"`	"current page"	"current page"	"current page"
`aria-disabled="true"`	"unavailable"	"unavailable"	"dimmed"

---

12. Anti-Patterns — What to Avoid

Focus management errors

Anti-pattern	What AT users experience	Fix
Focus on element with no name	Focus moves; screen reader announces silence	Add `role` + `aria-labelledby` before applying `tabindex="-1"`
Not returning focus after dialog close	Focus drops to `document.body`; user is disoriented	Store trigger reference; call `triggerRef.focus()` on close
`@if` removes focused element	Focus lands on `document.body`	Check `document.activeElement` before removing; move focus explicitly
`role="main"` on an Angular component	Two main landmarks confuse AT navigation	Use `role="region"` with `aria-label` on components

Role errors

Anti-pattern	Problem	Fix
`<div role="button">`	No keyboard support; Tab may not reach it	Use `<button>` — or add `tabindex="0"` + Enter/Space handlers
Redundant ARIA on native elements	`<a role="link">` — HTML already has the role	Remove — trust native semantics
`role="region"` with no accessible name	Region invisible in landmark list	Add `aria-label` or `aria-labelledby`
`role="none"` on focusable element	Element loses role but remains focusable; AT confused	Only use `role="none"` on non-interactive, non-focusable elements
Missing `aria-selected` on `role="option"`	AT cannot report selected state	Every option must have `aria-selected="true"` or `aria-selected="false"`

Labelling errors

Anti-pattern	AT experience	Fix
Icon button with no `aria-label`	"button, button, button" — no actionable context	`aria-label="Delete: [item title]"` + `aria-hidden="true"` on SVG
`aria-label` duplicating visible text	"Submit button, button" — role suffix announced twice	Use `aria-label="Submit"` not `aria-label="Submit button"`
`aria-labelledby` pointing to non-existent ID	Name is empty — AT announces role only	Validate all ID cross-references
`aria-hidden="true"` on a focusable element	Focus lands on element; AT says nothing	Remove focusability (`tabindex="-1"` → `-1` and `disabled`) or remove `aria-hidden`

Live region errors

Anti-pattern	Problem	Fix
Region created in same render cycle as content	AT misses the announcement	Always pre-render the region empty; change content inside it
`aria-live="assertive"` for all messages	Constant interruptions; AT user loses reading context	Use `polite` for informational; `assertive` only for destructive/critical events
`aria-atomic="false"` on count-change region	AT announces partial node ("4") without context	Add `aria-atomic="true"` — announces full region content
Route changes not announced in SPA	Page changes silently; AT user stranded	Implement `A11yTitleStrategy` with `LiveAnnouncer`

Angular-specific pitfalls

Pitfall	Root cause	Fix
OnPush + RxJS ARIA binding not updating	RxJS observable doesn't trigger signal-based CD	Use `computed()` signals; or `markForCheck()` in `.subscribe()`
Dynamic IDs breaking SSR hydration	`Math.random()` differs server vs client	Use deterministic IDs; inject `DOCUMENT` for SSR safety
`@defer` block exposing incomplete AT tree	Lazy content partially visible before fully loaded	Set `[attr.aria-busy]="true"` on parent region while deferred
CDK `FocusTrap` not destroyed	Memory leak; future traps may malfunction	Call `focusTrap.destroy()` in `ngOnDestroy`

---

13. Angular CDK — Installation & A11y Primitives

Installation

```
# Install Angular CDK (includes full a11y module)
npm install @angular/cdk

# Or use the Angular CLI schematic
ng add @angular/cdk

# Verify
npm list @angular/cdk
```

All CDK a11y services are `providedIn: 'root'` — no module registration required. Import into standalone components via `A11yModule` or direct service injection.

```
// Standalone component — import A11yModule for directives (cdkTrapFocus)
import { A11yModule } from '@angular/cdk/a11y';

@Component({
  imports: [A11yModule],
})
export class MyDialogComponent {}
```

CDK A11y primitives overview

Primitive	Type	Purpose
`cdkTrapFocus`	Directive	Trap keyboard focus inside a container (modal dialogs, drawers)
`cdkTrapFocusAutoCapture`	Input on directive	Automatically focus first element when trap activates
`FocusTrapFactory`	Service	Programmatically create/destroy focus traps
`LiveAnnouncer`	Service	Announce messages to screen readers via managed live region
`FocusMonitor`	Service	Track focus origin: `'keyboard'`, `'mouse'`, `'touch'`, `'program'`
`HighContrastModeDetector`	Service	Detect Windows Forced Colors / High Contrast mode
`AriaDescriber`	Service	Programmatically manage `aria-describedby` relationships
`ActiveDescendantKeyManager`	Service	Keyboard navigation for `aria-activedescendant` patterns
`CdkListbox` / `CdkOption`	Directives	Accessible listbox with roving tabindex and keyboard navigation
`CdkMenu` / `CdkMenuItem`	Directives	APG-compliant accessible menu and menu bar patterns

LiveAnnouncer

```
import { LiveAnnouncer } from '@angular/cdk/a11y';

@Component({
  /* standalone, OnPush */
})
export class TodoListComponent {
  private readonly live = inject(LiveAnnouncer);

  onToggleComplete(todo: Todo): void {
    const verb = todo.completed ? 'marked incomplete' : 'marked complete';
    this.todoService.toggle(todo.id);
    this.live.announce(`"${todo.title}" ${verb}`, 'polite');
  }
}
```

FocusTrap — directive vs service

```
<!-- Option A: directive (simplest) -->
<div
  role="dialog"
  aria-modal="true"
  aria-labelledby="dlg-title"
  cdkTrapFocus
  cdkTrapFocusAutoCapture
>
  <h2 id="dlg-title">{{ title() }}</h2>
  <button (click)="confirm()">Confirm</button>
  <button (click)="cancel()">Cancel</button>
</div>
```

```
// Option B: programmatic (full lifecycle control)
import { FocusTrapFactory, FocusTrap } from '@angular/cdk/a11y';

@Component({
  /* ... */
})
export class DrawerComponent implements OnDestroy {
  private readonly ftFactory = inject(FocusTrapFactory);
  private readonly elRef = inject(ElementRef<HTMLElement>);
  private focusTrap: FocusTrap | null = null;

  open(): void {
    this.focusTrap = this.ftFactory.create(this.elRef.nativeElement);
    this.focusTrap.focusInitialElementWhenReady();
  }

  ngOnDestroy(): void {
    this.focusTrap?.destroy();
  }
}
```

FocusMonitor — keyboard vs mouse focus rings

```
import { FocusMonitor, FocusOrigin } from '@angular/cdk/a11y';

@Component({
  selector: 'app-custom-button',
  template: ` <button #btn type="button" [class.ring-2]="focusOrigin() === 'keyboard'">
    <ng-content />
  </button>`,
})
export class CustomButtonComponent implements OnDestroy {
  readonly btnRef = viewChild<ElementRef>('btn');
  readonly focusOrigin = signal<FocusOrigin>(null);
  private readonly fm = inject(FocusMonitor);

  constructor() {
    afterNextRender(() => {
      this.fm.monitor(this.btnRef()!.nativeElement).subscribe((o) => this.focusOrigin.set(o));
    });
  }

  ngOnDestroy(): void {
    this.fm.stopMonitoring(this.btnRef()!.nativeElement);
  }
}
// CDK also auto-adds .cdk-keyboard-focused CSS class — usable in stylesheets directly
```

HighContrastModeDetector

```
import { HighContrastModeDetector } from '@angular/cdk/a11y';

@Component({
  /* ... */
})
export class StatusBadgeComponent {
  private readonly hcd = inject(HighContrastModeDetector);
  // Returns 'black-on-white' | 'white-on-black' | 'none'
  readonly isHighContrast = this.hcd.getHighContrastMode() !== 'none';
}
```

```
/* Prefer CSS media query over JS detection where possible */
@media (forced-colors: active) {
  .status-badge {
    border: 2px solid ButtonText;
  }
}
```

Signal-based ARIA in Angular 21

```
@Component({
  selector: 'app-todo-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  host: {
    role: 'region',
    '[attr.aria-label]': 'ariaLabel()',
    '[attr.aria-busy]': 'isLoading()',
  },
})
export class TodoListComponent {
  protected readonly filter = signal<'all' | 'active' | 'completed'>('all');
  protected readonly todos = signal<Todo[]>([]);
  protected readonly isLoading = signal(false);

  // Computed ARIA labels — auto-update on signal change
  protected readonly ariaLabel = computed(() => {
    const count = this.filtered().length;
    return `${this.filter()} tasks — ${count} item${count !== 1 ? 's' : ''}`;
  });

  // null removes the attribute; 'true' sets it
  protected readonly titleAriaInvalid = computed(() => (this.errors().title ? 'true' : null));
}
```

---

14. Testing Strategy

No single tool catches every accessibility issue. Effective testing requires all four layers.

The four-layer approach

Layer	Tools	Estimated WCAG coverage	When to run
Automated unit	axe-core + vitest-axe	~20–25%	Every test run in CI
Automated page audit	Lighthouse + axe DevTools + WAVE	~15–20% (overlap with unit)	Every pull request
Keyboard-only	Browser (no mouse)	~25%	Before every feature merge
Screen reader	NVDA + Firefox minimum	~30–35%	Before every release

> **Automated tools combined catch ~40% of real issues.** The remaining 60% require human testing. Both are non-negotiable.

axe-core unit test setup

```
npm install --save-dev axe-core vitest-axe
```

```
// component.spec.ts (Vitest)
import { axe, toHaveNoViolations } from 'vitest-axe';
expect.extend(toHaveNoViolations);

describe('TodoListComponent a11y', () => {
  it('passes axe WCAG scan', async () => {
    const fixture = TestBed.createComponent(TodoListComponent);
    fixture.detectChanges();
    const results = await axe(fixture.nativeElement);
    expect(results).toHaveNoViolations();
  });

  it('sets aria-invalid when title field has an error', () => {
    component.formErrors.set({ title: 'Required' });
    fixture.detectChanges();
    const input = fixture.nativeElement.querySelector('#task-title');
    expect(input.getAttribute('aria-invalid')).toBe('true');
  });

  it('binds null (not false) when field is valid', () => {
    component.formErrors.set({ title: '' });
    fixture.detectChanges();
    const input = fixture.nativeElement.querySelector('#task-title');
    expect(input.getAttribute('aria-invalid')).toBeNull();
  });

  it('announces deletion assertively', () => {
    const spy = vi.spyOn(liveAnnouncer, 'announce');
    component.onDelete({ id: '1', title: 'Buy milk', completed: false });
    expect(spy).toHaveBeenCalledWith(expect.stringContaining('deleted'), 'assertive');
  });
});
```

Chrome DevTools — accessibility tree inspection

1. `F12` → Elements → click **Accessibility** tab (in right pane)
2. Click any element → inspect: **Role**, **Name**, **Properties**, **States**
3. Enable "Show accessibility tree" to browse the full AT view

Lighthouse automated audit

1. `F12` → **Lighthouse** → tick **Accessibility only** → Analyse page load
2. Target: **90+ score**
3. Lighthouse checks: contrast ratios, form labels, alt text, valid ARIA, landmark presence, skip link, heading order

WAVE extension

- Install from Chrome Web Store
- Click WAVE icon on any page
- Red icons = errors; yellow = alerts; green = structural elements
- Useful for visualising landmark regions and heading hierarchy

Keyboard-only test protocol

Check	Pass / Fail criteria
Skip link	Appears on first Tab press; activates on Enter
Tab order	Logical left-to-right, top-to-bottom throughout the page
Focus visibility	Every focused element has a clearly visible focus indicator
All actions reachable	Forms, buttons, filters, delete, toggle — all without mouse
No keyboard traps	Tab out of every component is possible
Arrow keys in widgets	Work correctly in tabs, menus, listboxes
Escape key	Closes all dialogs and menus

NVDA screen reader test protocol

Check	Expected result
`NVDA+F7` landmarks	`main`, `navigation`, `contentinfo` + all named regions listed
`NVDA+F6` headings	Logical H1 → H2 → H3 hierarchy, no level skips
Tab to buttons	Each announces name + "button" (not just "button")
Form submission success	Status announcement without moving focus
Task deletion	Assertive announcement immediately interrupts
Filter change	Polite count announcement after current speech
Route navigation	Page title announced after navigation

---

15. Architecting an Angular 21 App for WCAG Compliance

WCAG conformance is an architectural concern, not a retrofit task. The sections below cover a phased approach — from project bootstrap through CI gating — and provide specific guidance for Angular Material, PrimeNG, and custom component libraries.

Architecture layers

Layer	Responsibility	Key tooling
Foundation	HTML semantics, `lang`, page `<title>`, skip link, viewport	`index.html`, `TitleStrategy`
Component	ARIA integration, keyboard contracts, focus management	`@angular/cdk/a11y`, CDK primitives
Application	Route announcements, live regions, form validation errors	`LiveAnnouncer`, `A11yTitleStrategy`
Quality	Automated regression, manual review, CI gate	`axe-core`, Lighthouse CI, NVDA

Phase 1 — Project bootstrap checklist

#	Step	File(s) affected	WCAG criterion
1	Add `lang="en"` to `<html>`	`index.html`	3.1.1 (A)
2	Set meaningful `<title>` per route via `TitleStrategy`	`app.config.ts`, `app.routes.ts`	2.4.2 (A)
3	Insert skip link as the first focusable element in `<body>`	`app.html`	2.4.1 (A)
4	Set `<meta name="viewport" content="width=device-width, initial-scale=1">`	`index.html`	1.4.4 (AA)
5	Verify body text/background contrast ≥ 4.5:1	`styles.css` / Tailwind config	1.4.3 (AA)
6	Install `@angular/cdk` — `ng add @angular/cdk`	`package.json`	All focus/announce requirements
7	Import `A11yModule` and `provideAnimationsAsync`	`app.config.ts`	—
8	Wire `A11yTitleStrategy` for SPA route announcements	`app.config.ts`	2.4.2 + 3.2.3

Skip link implementation:

```
<!-- app.html — first element in <body> -->
<a href="#main-content" class="sr-only focus:not-sr-only focus:absolute focus:top-2 focus:left-2">
  Skip to main content
</a>
<main id="main-content" tabindex="-1">
  <router-outlet />
</main>
```

`A11yTitleStrategy` — SPA route announcement:

```
// a11y-title.strategy.ts
@Injectable({ providedIn: 'root' })
export class A11yTitleStrategy extends TitleStrategy {
  private title = inject(Title);
  private announcer = inject(LiveAnnouncer);

  override updateTitle(snapshot: RouterStateSnapshot): void {
    const pageTitle = this.buildTitle(snapshot) ?? 'Angular App';
    this.title.setTitle(pageTitle);
    this.announcer.announce(`Navigated to ${pageTitle}`, 'polite');
  }
}

// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideAnimationsAsync(),
    { provide: TitleStrategy, useClass: A11yTitleStrategy },
  ],
};
```

Phase 2 — Component architecture rules

Rule	Correct Angular 21 pattern	Anti-pattern
One `<main>` per page	Shell layout owns `<main>`; feature components use `<section>` or `role="region"`	Feature component adds `host: { 'role': 'main' }`
Landmark regions are labelled	`aria-label` or `aria-labelledby` on every `<nav>`, `<aside>`, `<section>`	Bare `<nav>` with no label
Icon buttons carry descriptive labels	`[attr.aria-label]="'Delete: ' + item().title"`	`<button><svg>…</svg></button>`
Focus returns after dialog close	Store opener ref; call `.focus()` in dialog close handler	Dialog closes; focus drops to `<body>`
ARIA attributes bound with signals	`[attr.aria-invalid]="invalid() ? true : null"`	Hard-coded `aria-invalid="false"`
No `aria-hidden` on focusable elements	`aria-hidden="true"` only on decorative elements that are not in the tab order	`aria-hidden="true"` on a button

Phase 3 — Signal-based ARIA service (Angular 21)

```
// a11y-state.service.ts
@Injectable({ providedIn: 'root' })
export class A11yStateService {
  private announcer = inject(LiveAnnouncer);
  readonly formErrors = signal<Record<string, string>>({});

  announce(message: string, politeness: AriaLivePoliteness = 'polite'): void {
    this.announcer.announce(message, politeness);
  }

  // Returns error element id only when an error exists for the field
  describedBy(fieldId: string): string | null {
    return this.formErrors()[fieldId] ? `${fieldId}-error` : null;
  }

  setError(fieldId: string, message: string): void {
    this.formErrors.update((errs) => ({ ...errs, [fieldId]: message }));
  }

  clearError(fieldId: string): void {
    this.formErrors.update(({ [fieldId]: _, ...rest }) => rest);
  }
}
```

```
// todo-form.component.ts — usage
@Component({
  selector: 'app-todo-form',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <label for="todo-input">Task title</label>
    <input
      id="todo-input"
      [attr.aria-invalid]="titleInvalid() ? true : null"
      [attr.aria-describedby]="a11y.describedBy('todo-input')"
      [attr.aria-required]="true"
    />
    <span id="todo-input-error" aria-live="polite">
      @if (titleInvalid()) {
        Title is required
      }
    </span>
  `,
})
export class TodoFormComponent {
  a11y = inject(A11yStateService);
  titleInvalid = computed(() => /* validation signal */ false);
}
```

Phase 4 — Focus management strategy

```
Route change
  ├─ TitleStrategy sets <title> + announces via LiveAnnouncer
  └─ Focus moves to <main tabindex="-1"> (router scroll strategy)

Modal / Dialog opens
  ├─ [attr.inert]="" applied to <main> (blocks background interaction)
  ├─ cdkTrapFocus activated on dialog host
  └─ cdkTrapFocusAutoCapture moves focus to first interactive element

Modal / Dialog closes
  ├─ inert attribute removed from <main>
  └─ opener element reference.focus() called explicitly
```

```
// dialog host element
@Component({
  template: `
    <div
      cdkTrapFocus
      cdkTrapFocusAutoCapture
      role="dialog"
      [attr.aria-labelledby]="titleId()"
      aria-modal="true"
    >
      <h2 [id]="titleId()">{{ title() }}</h2>
      <ng-content />
      <button (click)="close()">Close</button>
    </div>
  `,
})
export class DialogComponent {
  private openerRef = signal<HTMLElement | null>(null);

  open(opener: HTMLElement): void {
    this.openerRef.set(opener);
  }

  close(): void {
    this.openerRef()?.focus(); // return focus to opener
  }
}
```

Phase 5 — Live region architecture

Pre-render all live regions in the root shell. Never create them conditionally.

```
<!-- app.html — always in DOM before any dynamic content -->
<div aria-live="polite" aria-atomic="true" class="sr-only" id="status-region"></div>
<div aria-live="assertive" aria-atomic="true" class="sr-only" id="alert-region"></div>
```

Action type	Politeness	Mechanism
CRUD success (add / update / delete)	`polite`	`LiveAnnouncer.announce()`
Filter / sort applied	`polite`	`LiveAnnouncer.announce()`
Form validation error	`assertive`	Inline `role="alert"` / `aria-live="assertive"`
Session timeout warning	`assertive`	`LiveAnnouncer.announce()`
Route change	`polite`	`A11yTitleStrategy`

Phase 6 — Reusable form field pattern

```
// lib-field.component.ts
@Component({
  selector: 'lib-field',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <label [for]="id()">{{ label() }}</label>
    <ng-content />
    <span [id]="errorId()" aria-live="polite" class="error">
      @if (isInvalid()) {
        {{ errorMessage() }}
      }
    </span>
  `,
})
export class FieldComponent {
  id = input.required<string>();
  label = input.required<string>();
  control = input.required<AbstractControl>();
  errorMessage = input<string>('This field is invalid');

  errorId = computed(() => `${this.id()}-error`);
  isInvalid = computed(() => this.control().invalid && this.control().touched);
}
```

Inner `<input>` binds the computed IDs from the parent `lib-field`:

```
<input
  [id]="fieldId"
  [attr.aria-invalid]="field.isInvalid() ? true : null"
  [attr.aria-describedby]="field.isInvalid() ? field.errorId() : null"
  [attr.aria-required]="required ? true : null"
/>
```

Working with UI libraries

Angular Material

Angular Material implements CDK a11y primitives internally but requires explicit configuration for full WCAG conformance:

Component	Required action	Default gap
`mat-form-field`	Always provide `<mat-label>`	Without it: no accessible name on the input
`mat-icon-button`	Add `matTooltip` + `aria-label`	`<mat-icon>` alone has no text for AT
`mat-dialog`	Set `aria-labelledby` pointing to dialog title element	Default dialog has no accessible name
`mat-select`	Works natively — ensure `<mat-label>` is present	Passes AA with label; fails without
`mat-table`	Add `aria-label` to `<mat-table>`; sort headers emit `aria-sort` automatically	Unlabelled tables fail 1.3.1
`mat-menu`	Keyboard contract is built-in; add `aria-label` to trigger if icon-only	—
`mat-stepper`	Step labels must be meaningful text — not icon-only	Icon-only steps fail 1.1.1
`mat-slide-toggle`	Requires `<mat-label>` or `aria-label` input	Default has no name
`mat-chip`	Add `aria-label` on remove button	Default remove icon is unlabelled

Global Material setup for WCAG AA:

```ts
// app.config.ts
import { MAT_FORM_FIELD_DEFAULT_OPTIONS } from '@angular/material/form-field';

export const appConfig: ApplicationConfig = {
  providers: [
    provideAnimationsAsync(),
    {
      provide: MAT_FORM_FIELD_DEFAULT_OPTIONS,
      useValue: { appearance: 'outline', floatLabel: 'always' },
      // floatLabel:'always' keeps the label visible at all times —
      // prevents placeholder-as-label WCAG 1.3.5 / 3.3.2 failure
    },
  ],
};
```

PrimeNG

PrimeNG v17+ has improved ARIA support. Components still require explicit input bindings for full conformance:

Component	Required action	Notes
`p-button` (icon-only)	`[ariaLabel]="'Delete: ' + item.title"`	`icon` prop alone is invisible to AT
`p-dialog`	Set `[header]="dialogTitle"` + `[ariaCloseLabel]="'Close dialog'"`	PrimeNG sets `role="dialog"` and `aria-modal` automatically
`p-table`	Set `[caption]="tableSummary"`	Required for WCAG 1.3.1 on data tables
`p-dropdown`	Always provide outer `<label [for]="inputId">`	Placeholder is not an accessible label
`p-inputtext`	Bind `[ariaDescribedBy]="errorId"` when error is shown	Native `<input>` — bind exactly as any HTML input
`p-toast`	Default `error` severity uses `assertive` — use `info` / `success` for non-urgent messages	Match politeness to urgency
`p-menu` / `p-tieredmenu`	Add `[ariaLabel]` to distinguish multiple menus on one page	—
`p-checkbox`	Requires `[label]` prop or associated `<label>` element	Default has no visible label
`p-datepicker`	Ensure `[inputId]` and outer `<label [for]="inputId">` are set	—

```ts
// PrimeNG provider setup (v17+)
import { providePrimeNG } from 'primeng/config';
import Aura from '@primeng/themes/aura';

export const appConfig: ApplicationConfig = {
  providers: [providePrimeNG({ theme: { preset: Aura } })],
};
```

Custom UI component library

When building or consuming an internal design system, enforce accessibility at the library boundary:

Layer	Enforcement mechanism
Component contract	Document required vs optional ARIA inputs; use `input.required<string>()` for `ariaLabel` when no visible text exists
Base interactive class	Extend from a base class that injects `FocusMonitor` and exposes a focus-origin signal
Design tokens	Enforce min 4.5:1 contrast ratio in the token set; provide a `high-contrast` theme variant using `HighContrastModeDetector`
Storybook	Add `@storybook/addon-a11y` to every story; configure axe rules to fail on `wcag2a`, `wcag2aa`
Per-component checklist	Ship an accessibility checklist alongside each component in the library docs

Skeleton — accessible custom button:

```
@Component({
  selector: 'lib-button',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <button
      [type]="type()"
      [disabled]="disabled()"
      [attr.aria-label]="ariaLabel() ?? null"
      [attr.aria-busy]="loading() ? true : null"
      [attr.aria-pressed]="pressed() ?? null"
    >
      @if (loading()) {
        <span aria-hidden="true" class="spinner"></span>
        <span class="sr-only">Loading…</span>
      }
      <ng-content />
    </button>
  `,
})
export class LibButtonComponent {
  type = input<'button' | 'submit' | 'reset'>('button');
  disabled = input<boolean>(false);
  loading = input<boolean>(false);
  pressed = input<boolean | null>(null); // for toggle buttons
  ariaLabel = input<string | null>(null); // required when no visible text
}
```

Phase 7 — CI conformance gate

Stage	Tool	Pass threshold
Local dev	axe DevTools browser extension	Zero critical / serious violations
Pre-commit	`angular-eslint` a11y rules	Fail on any error
CI unit tests	`axe-core` + `@angular/cdk/testing`	Zero violations
CI E2E	`cypress-axe`	Zero critical
CI Lighthouse	Lighthouse CI Action	Accessibility score ≥ 95
Release gate	Manual NVDA + keyboard review	Passes full keyboard test protocol

Lighthouse CI budget file:

```
[{ "resourceType": "accessibility", "budget": 95 }]
```

GitHub Actions excerpt:

```yaml
- name: axe CLI audit
  run: |
    npx @axe-core/cli http://localhost:4200 \
      --exit \
      --tags wcag2a,wcag2aa,wcag21aa

- name: Lighthouse accessibility score
  uses: treosh/lighthouse-ci-action@v10
  with:
    urls: http://localhost:4200
    budgetPath: ./a11y-budget.json
```

---

16. Quick Reference Cheat Sheet

Naming decision tree

```
Has visible text that describes it?
  YES → Let text content be the name (buttons, links)
  NO  → Is it a form field?
          YES → Use <label for="id">
          NO  → Use aria-label="string"
              → Or aria-labelledby="heading-id" if heading is nearby
```

The 15 rules that prevent 90% of failures

#	Rule
1	Use semantic HTML first — every correct native element is a free ARIA implementation
2	One `<main>` per page — components use `role="region"`, never `role="main"`
3	Skip link must be the **first focusable element** — WCAG 2.4.1 Level A
4	Every icon button needs `aria-label="Action: Item name"` — not just "Delete"
5	Bind `aria-invalid="true"` on validation failure — red colour alone is invisible to AT
6	Bind `null` to remove ARIA attributes — not `'false'`
7	Live regions must pre-exist in the DOM before their content is injected
8	`aria-live="polite"` for information; `assertive` only for destructive / critical
9	Set `aria-atomic="true"` on regions where partial text lacks context
10	SPA route changes require explicit announcement (use `A11yTitleStrategy`)
11	Install `@angular/cdk` — `LiveAnnouncer`, `FocusTrap`, `FocusMonitor` are ready-made
12	Consult the APG before building any custom widget — follow the keyboard contract exactly
13	Return focus to the opener element when a dialog closes
14	Never put `aria-hidden="true"` on a focusable element — focus becomes silent
15	Test with keyboard first, then NVDA + Firefox — no automated tool replaces human review

ARIA at a glance

Need	Use
Name a control without visible text	`aria-label="…"`
Name a region using its heading	`aria-labelledby="heading-id"`
Add description to a field	`aria-describedby="desc-id"`
Mark a field required	`aria-required="true"`
Mark a field invalid	`aria-invalid="true"` + `aria-describedby` pointing to error
Show expanded / collapsed	`aria-expanded="true|false"`
Show checked / unchecked	`aria-checked="true|false|mixed"`
Show which tab / nav link is current	`aria-selected="true"` / `aria-current="page"`
Announce on content change	`aria-live="polite|assertive"` + `aria-atomic="true"`
Keep focus in dialog	`cdkTrapFocus` + `cdkTrapFocusAutoCapture`
Block background when modal open	`[attr.inert]="isModalOpen() ? '' : null"` on `<main>`
Hide decorative icon from AT	`aria-hidden="true"` on `<svg>`

---

17. Resources

Standards and specifications

Resource	URL
WCAG 2.1 full specification	w3.org/TR/WCAG21
WCAG 2.2 (what's new)	w3.org/TR/WCAG22
WAI-ARIA 1.2 specification	w3.org/TR/wai-aria-1.2
ARIA Authoring Practices Guide	w3.org/WAI/ARIA/apg/patterns
HTML Accessibility API Mappings	w3.org/TR/html-aam-1.0

Angular resources

Resource	URL
Angular accessibility guide	angular.dev/best-practices/a11y
Angular CDK a11y module	material.angular.io/cdk/a11y
axe-core (automated WCAG)	github.com/dequelabs/axe-core
Source code for this guide	github.com/sudeep31/angularTodo — documents branch

Free testing tools

Tool	Where to get it
NVDA screen reader (Windows)	nvaccess.org/download
WAVE accessibility evaluator	wave.webaim.org
axe DevTools extension	Chrome Web Store: "axe devtools"
Colour Contrast Analyser	tpgi.com/color-contrast-checker
Accessibility Insights (MS)	accessibilityinsights.io

Further learning

Resource	URL
A11y Project checklist	a11yproject.com/checklist
WebAIM screen reader survey	webaim.org/projects/screenreadersurvey10
Deque University (free resources)	dequeuniversity.com
Inclusive Design Principles	inclusivedesignprinciples.org
MDN ARIA docs	developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA

#WebAccessibility #A11y #WCAG #ARIA #Angular #AngularCDK #FrontendDevelopment #InclusiveDesign